Innomatics Research Labs

A LOW-LEVEL DESIGN (LLD) REPORT
ON

"RAG-BASED CUSTOMER SUPPORT ASSISTANT"

A Project Report Submitted in Partial Fulfillment of the Requirements for the Agentic AI Internship

Submitted by:

Sushil Panthi
Intern ID: IN226018202

Guided by:
Mr. Mayank Ghai
Trainer | Mentor
Innomatics Research Lab

Innomatics Research Labs

DECLARATION

I, Sushil Panthi, hereby declare that the project entitled "RAG-Based Customer Support Assistant using LangGraph and Human-in-the-Loop (HITL)" is an original work carried out by me as part of the GenAI Internship at Innomatics Research Labs.

This Low-Level Design (LLD) document provides the technical implementation details, including the ChromaDB storage module, LangGraph state transitions, and the internal logic for HITL escalation. All modules and data structures have been designed to meet the production-style requirements of the assignment.

Sushil Panthi
Date: April 23, 2026
Location: Nepal

Innomatics Research Labs

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Innomatics Research Labs for providing this challenging and insightful internship opportunity. This project has allowed me to apply theoretical knowledge of Generative AI and Agentic Frameworks into a production-style workflow.

Special thanks to my mentor, Mr. Mayank Ghai, for the technical guidance provided during this project. Also, to the technical team for designing a curriculum that emphasizes system design thinking. Designing the internal logic for an agentic RAG system has provided deep insights into modular software engineering and the practical challenges of Human-AI collaboration.

I also acknowledge the open-source community behind LangChain, LangGraph, and ChromaDB, whose robust APIs form the backbone of this implementation.

Abstract

This Low-Level Design (LLD) document provides a comprehensive technical blueprint for the implementation of the RAG-Based Customer Support Assistant. While the High-Level Design (HLD) established the structural boundaries, this report details the internal mechanics of the system's modular components .

The implementation features a dedicated Document Processing Module that utilizes a recursive chunking strategy (1000 characters with 100-character overlap) to prepare data for the Vector Storage Module. Central to the system is a locally persisted ChromaDB collection, which manages high-dimensional embeddings for efficient semantic retrieval .

A critical technical highlight is the application of LangGraph to manage the agentic state transitions. Unlike linear pipelines, this design defines a specific State Object to track the query, retrieved context, and confidence scores across nodes. The control flow incorporates Conditional Routing Logic, specifically engineered to handle HITL (Human-in-the-Loop) escalation. Escalation is triggered by deterministic criteria, such as low confidence thresholds (< 0.7) or context gaps, and is accompanied by dynamically generated escalation reasons .

The document further outlines the modular prompt architecture, API schemas, and the Streamlit interaction flow, which includes clickable demo questions and real-time processing states to ensure a professional user experience . By focusing on modularity and traceability via LangSmith, this design ensures a robust, production-grade deployment.

Table of Content

Chapter No.	Chapter / Sub-Chapter Name		Page No
1	Module-Level Design		1-3
	1.1	Configuration & Environment Module	1
	1.2	Modular Prompt Management Directory	1
	1.3	Document Processing & Ingestion Module	2
	1.4	Internal Module Interaction	2
2	Data Structures & Schemas		4
	2.1	LangGraph State Object Definition	4
	2.2	Document Chunk & Metadata Schema	4
	2.3	API Input/Output Response Schema	4
3	Vector Storage Module (ChromaDB)		5-7
	3.1	ChromaDB Collection & Local Persistence	5
	3.2	Embedding Generation Logic	5
	3.3	Retrieval & Similarity Thresholding	6
	3.4	Vector Storage Implementation Summary	6
4	Graph Execution & Control Flow		8-9
	4.1	Node-by-Node Functional Logic	8
	4.2	Conditional Routing Implementation	8
	4.3	Iterative Loop & State Transitions	8
5	Escalation & HITL Logic		10-11
	5.1	Automated Escalation Reason Generation	10
	5.2	Human-in-the-Loop Integration Points	10
6	User Interface & Interaction		12-13
	6.1	Streamlit Workflow & Processing States	12
	6.2	Clickable Demo Questions & Sidebar	12
	6.3	UI Layout Summary	12
7	Error Handling & Observability		14
	7.1	Exception Management & Model Fallbacks	14
	7.2	Traceability Hooks (LangSmith)	14
8	Conclusion		15
9	Bibliography		16

List of Figure

Fig.No	Figure Name	Page No
1	Internal Module interaction showing the decoupling of configuration, prompt templates, and core agentic logic	3
2	ChromaDB Local Persistence Schema illustrating the mapping of Document Chunks to Vector Embeddings and Metadata	6
3	Low-Level State Transition Map tracking the 'AgentState' object through Retrieval, Generation, and Escalation nodes	9
4	Decision Tree for HITL Escalation: Logic paths for generating specific escalation reasons	11
5	Streamlit Frontend Layout highlighting the sidebar for demo questions and real-time processing status indicators	12

CHAPTER 1: MODULE-LEVEL DESIGN

This chapter provides a granular breakdown of the internal software architecture, focusing on the decoupling of logic, configuration, and data processing to ensure a production-grade, maintainable system.

1.1 Configuration & Environment Module (config.py)

The configuration module serves as the centralized "Source of Truth" for the application's environment and hyperparameters. It abstracts sensitive and variable parameters away from the core execution logic.

a) **Environment Management:** Utilizes the python-dotenv library to securely load API keys for Groq and Google Gemini, along with LangSmith project credentials for observability.
b) **Hyperparameter Definition:** Sets global variables for the RAG pipeline, including:
    - CHUNK_SIZE: 1,000 characters.
    - CHUNK_OVERLAP: 100 characters.
    - MODEL_PROVIDER: Defines "Groq" as the primary inference engine and "Gemini" as the secondary fallback.
c) **Database Paths:** Explicitly defines the CHROMA_PERSIST_DIR to ensure that the vector database maintains its state on the local disk between application restarts.

1.2 Modular Prompt Management Directory (prompts/)

To prevent technical debt and allow for rapid iteration of the assistant's "personality" and accuracy, all instructions are stored as modular prompt files rather than being hardcoded into the functions .

a) **Prompt Separation:** The directory contains standalone templates (e.g., templates.py) that house specific ChatPromptTemplate objects.
b) **Intent Detection Template:** A specialized prompt designed to categorize user queries before they hit the RAG pipeline.
c) **Grounded RAG Template:** A highly structured instruction that forces the LLM to only answer using the retrieved context, effectively preventing hallucinations.

d) **Escalation Template:** Formats the "Handover Packet" when a query is routed to human support, ensuring the human agent receives a clear summary of the AI's failure reason.

1.3 Document Processing & Ingestion Module (ingestion.py)

This module is responsible for the technical lifecycle of transforming a static PDF into a searchable vector index within ChromaDB .

a) **PDF Loading Logic:** Implements the PyPDFLoader to iterate through the target document, extracting clean text while capturing essential metadata such as page numbers.

b) **Chunking Strategy:** Utilizes the RecursiveCharacterTextSplitter. This algorithm intelligently splits text by looking for natural breaks (like double newlines or periods) to ensure that chunks do not break mid-sentence, preserving semantic continuity.

c) **Embedding Interface:** Connects to the GoogleGenerativeAIEmbeddings model to transform the 1,000-character text segments into 768-dimensional mathematical vectors.

d) **Vector Storage Interaction:** Coordinates with the **Vector Storage Module** to upsert these embeddings into a local ChromaDB collection, followed by a mandatory .persist() command to lock the data into local storage.

1.4 Internal Module Interaction

The operational efficiency of the system depends on the seamless coordination between specialized modules. As visualized in below Figure 1, these modules do not operate in silos but are orchestrated by the LangGraph Framework to ensure a stateful execution cycle .

- **Orchestration Logic:** The workflow.py script acts as the central hub, pulling environmental variables from config.py and structured instructions from the prompts/ directory .
- **Data Synchronization:** During the 2-node flow, the retrieval_node triggers the vector_store.py module to fetch document chunks, while the generation_node utilizes the Modular Prompt Files to synthesize the final grounded output .
- **Safety Interdependency:** The HITL Module relies on the routing_node to

monitor the State Object; if the confidence thresholds defined in config.py are breached, the system halts automated execution and passes the context to the human-interaction layer .

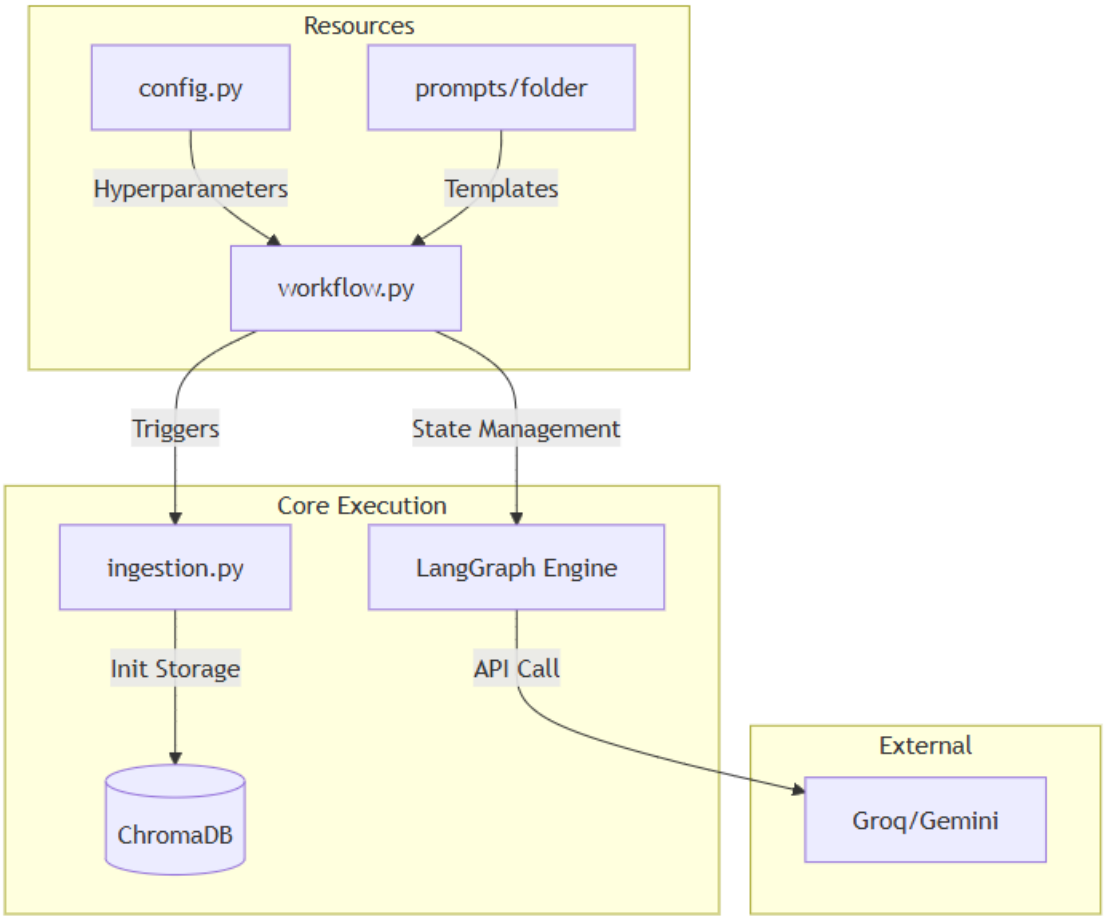

Figure 1 Internal Module interaction showing the decoupling of configuration, prompt templates, and core agentic logic

CHAPTER 2: DATA STRUCTURES & SCHEMAS

This chapter defines the technical specifications for the data that flows through the system, ensuring consistency and type-safety across the agentic lifecycle .

2.1 LangGraph State Object Definition

The core of the system is the State Object, implemented as a TypedDict. This object maintains the "memory" of the agent as it transitions between nodes.

- **question (str):** The immutable user query captured at the input node.
- **context (str):** The aggregated text chunks retrieved from ChromaDB.
- **answer (str):** The final generated response or the human-verified output.
- **confidence (float):** A numeric score (0.0 to 1.0) representing the LLM's certainty in its response.
- **escalation_reason (str):** A dynamically generated string explaining why a hand-off occurred.

2.2 Document Chunk & Metadata Schema

When storing data in the Vector Storage Module, a strict schema is followed to ensure precise retrieval :

- **id:** A unique hash generated for each 1,000-character chunk.
- **text:** The raw content segment extracted from the PDF.
- **metadata:** A dictionary containing the source (file name) and page_number to allow the assistant to cite specific pages in the final answer.

2.3 API Input/Output Response Schema

To ensure compatibility with front-end interfaces, the system communicates using a structured JSON-style schema :

- **Input Schema:** {"user_id": str, "query": str, "mode": "RAG"}.
- **Output Schema:** {"status": "success/escalated", "answer": str, "citations": list[int], "reason": str | null}.

CHAPTER 3: VECTOR STORAGE MODULE (CHROMADB)

This chapter provides a technical deep-dive into the specialized module responsible for the system's long-term memory and local data persistence. To achieve the mandatory project requirements, this module is engineered to handle sensitive data locally while providing high-speed semantic retrieval.

3.1 ChromaDB Collection & Local Persistence

The system interacts with a dedicated ChromaDB instance configured for local execution. Unlike cloud-based vector stores, this implementation emphasizes data sovereignty and low latency .

- **Collection Management:** A collection named support_knowledge is initialized to store document embeddings and their associated metadata.
- **Local Storage Behavior:** The PersistentClient is used with a specific persist_directory (defined in config.py), ensuring that all indexed vectors are saved to the local file system.
- **Data Integrity:** The module includes a validation check to ensure the collection exists before attempting a query; if the collection is empty, it triggers a warning to the Ingestion Module.

3.2 Embedding Generation Logic

The transformation of text chunks into mathematical vectors is handled through a standardized interface to ensure consistency.

- **Model Integration:** The module utilizes the GoogleGenerativeAIEmbeddings class, specifically the models/embedding-001 variant.
- **Vector Dimensionality:** Each 1,000-character chunk is mapped to a 768-dimensional vector space, capturing complex semantic relationships rather than simple keyword frequencies.
- **Batch Processing:** To optimize performance during the **Ingestion Phase**, embeddings are generated in batches, reducing the number of API calls to the provider.

3.3 Retrieval & Similarity Thresholding

The retrieval logic is designed to be "risk-aware," filtering results based on mathematical relevance to prevent the generation of inaccurate answers .

- **Cosine Similarity Search:** When a query is received, it is vectorized and compared against the ChromaDB collection using a cosine similarity metric.
- **Top-K Retrieval:** The module is configured to return the top 3-5 most relevant chunks (the k parameter) to provide the LLM with sufficient context.
- **Threshold-Based Gating:** A similarity threshold (e.g., 0.5) is applied to the retrieved results. If the highest-ranking chunk fails to meet this threshold, the module flags the State Object as having "Insufficient Context," which subsequently triggers the HITL Escalation path.

3.4 Vector Storage Implementation Summary

The technical flow within this module follows the mandatory sequence:

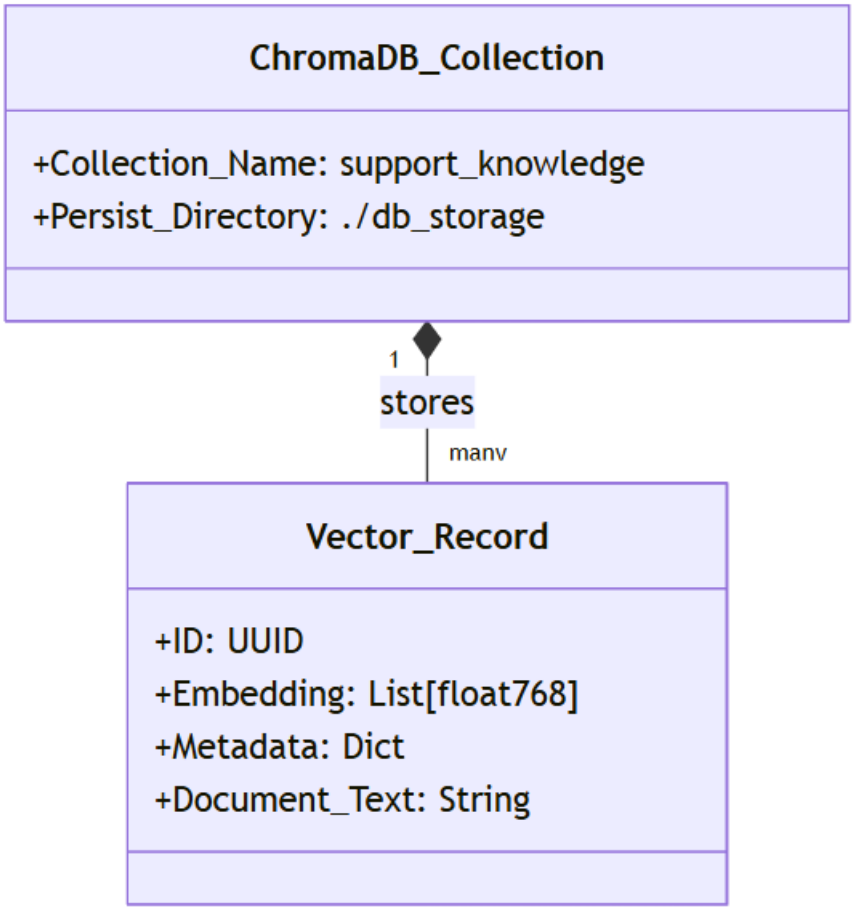

Figure 2 ChromaDB Local Persistence Schema illustrating the mapping of Document Chunks to Vector Embeddings and Metadata

Chunked Text → Google Embedding Model → ChromaDB Vector Store → Similarity-Based Retrieval.

The above technical flow within this module is engineered to ensure data integrity and retrieval precision. By following a strictly local persistence model, the system satisfies production-style security requirements while maintaining high-speed access to the knowledge base .

The implementation follows this mandatory technical sequence:

- **Vectorization:** Each 1,000-character chunk is processed by the GoogleGenerativeAIEmbeddings interface to produce a high-dimensional vector.
- **Indexing:** These vectors are upserted into the support_knowledge collection within the ChromaDB Vector Store.
- **Persistence:** The module invokes the .persist() method, ensuring the ChromaDB Local Storage behavior saves the index to the defined directory on the disk.
- **Thresholded Retrieval:** Upon a user query, the system identifies relevant vectors and applies a similarity filter to ensure only high-confidence context is extracted for the final answer.

CHAPTER 4: GRAPH EXECUTION & CONTROL FLOW

This chapter details the operational logic of the LangGraph Orchestrator, defining how the system manages state transitions and conditional routing between nodes.

4.1 Node-by-Node Functional Logic

The agentic workflow is broken down into discrete functional units (nodes), each with a specialized responsibility in the query lifecycle :

- **retrieve_node**: This function serves as the interface to the ChromaDB module. It vectorizes the user's question, queries the local collection, and updates the context key in the State Object with the top-ranked text segments.
- **generate_node**: This node utilizes the Modular Prompt Files and the retrieved context. It passes this data to the Groq LPU (or Gemini fallback) to synthesize a grounded response.
- **human_support_node**: A specialized node that acts as a "waiting room" for queries that fail the automated checks. It bundles the question with a generated Escalation Reason for human review .

4.2 Conditional Routing Implementation

The intelligence of the workflow is governed by the routing_node, which acts as a traffic controller based on the current state.

- **Confidence Logic**: The system checks the confidence score generated by the LLM. If the score is >= 0.7, the graph routes to the END.
- **Failure Logic**: If the confidence is < 0.7 or if the context is empty, the router redirects the flow to the HITL Escalation node.

4.3 Iterative Loop & State Transitions

The LangGraph State Transition Map ensures that the interaction remains stateful and controlled throughout the query lifecycle. Unlike standard linear chains, the iterative nature of the graph allows for complex decision-making before reaching a terminal state.

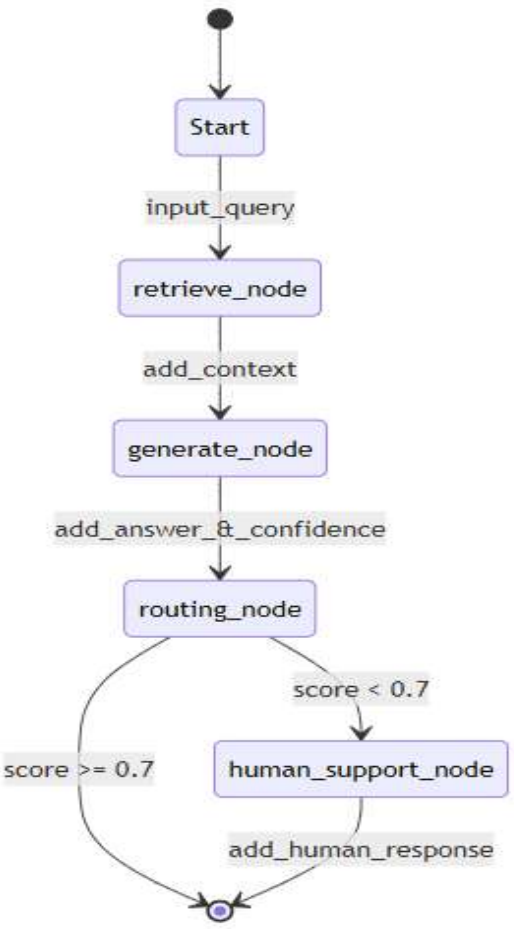

Figure 3 Low-Level State Transition Map tracking the 'AgentState' object through Retrieval, Generation, and Escalation nodes

- **State Persistence:** The AgentState object is passed through each node, where it is updated rather than replaced, preserving the history of retrieval and generation attempts.
- **Edge Logic:** Transitions are governed by conditional edges that evaluate the current state variables (such as confidence and context) to determine the next destination.
- **Termination Paths:** The graph only exits when the routing logic reaches the END node (successful response) or completes the HITL Escalation path (human intervention), ensuring every process has a definitive conclusion.

CHAPTER 5: ESCALATION & HITL LOGIC

This chapter details the internal mechanics of the Human-in-the-Loop module, specifically how the system identifies the need for intervention and prepares data for a human agent.

5.1 Automated Escalation Reason Generation

To satisfy the mandatory requirement for transparency, the system does not escalate silently. It utilizes an internal Reasoning Engine to generate specific Escalation Reasons based on the failure state:

- **Low Confidence Trigger:** If the LLM self-evaluation returns a score below 0.7, the system generates: *"AI Confidence Threshold Not Met (Score: [Value])"*.
- **Knowledge Gap Trigger:** If ChromaDB returns no relevant results (K=0) or similarity scores are too low, the reason is recorded as: *"Inadequate documentation found in knowledge base"*.
- **Intent Mismatch:** If the user query is identified as out-of-scope or sensitive by the intent detector, the reason provided is: *"Sensitive or Restricted Query Detected"*.

5.2 Human-in-the-Loop Integration Points

The HITL module creates a seamless hand-off from AI to human by preparing an Escalation Packet:

- **Context Handoff:** The human agent is presented with the original question, the escalation_reason, and any partial context the AI was able to find.
- **State Suspension:** The graph execution enters a "breakpoint" state, preventing any further automated output until the human input is received.

Response Override: Once the human agent provides the verified answer, it is injected into the answer key of the State Object, which then moves to the final output node.

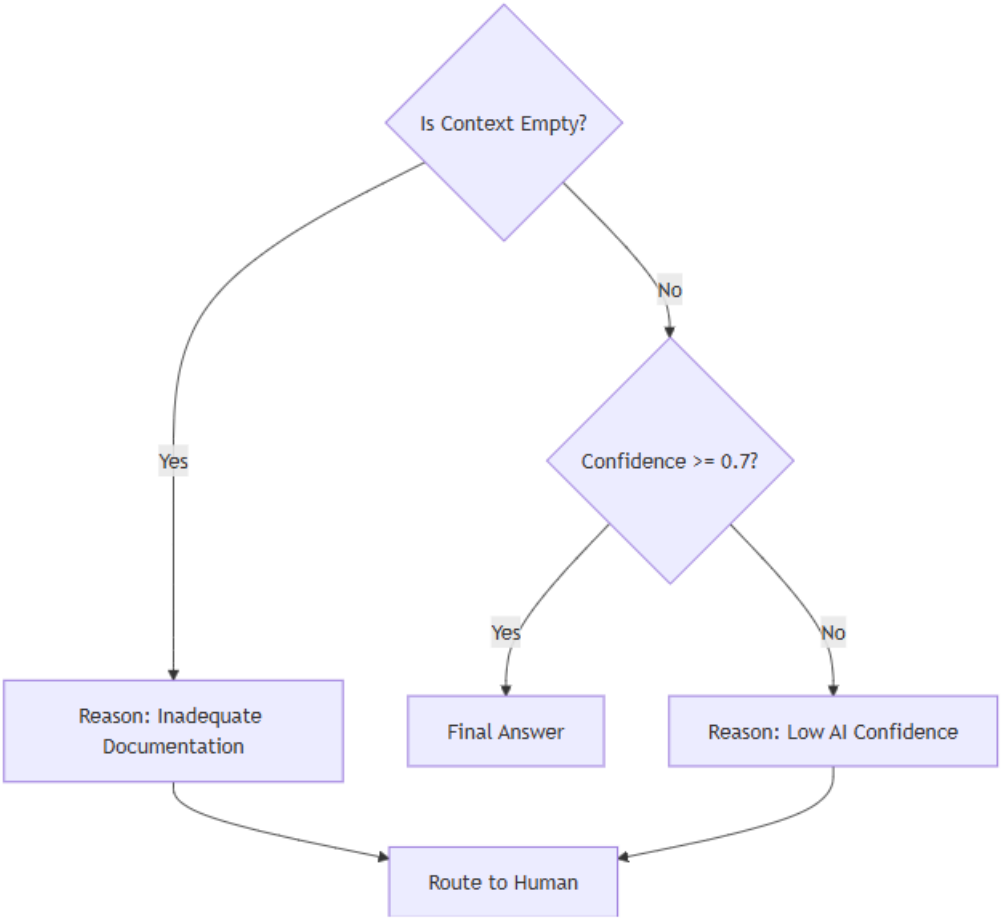

Figure 4 Decision Tree for HITL Escalation: Logic paths for generating specific escalation reasons

CHAPTER 6: USER INTERFACE & INTERACTION

This chapter outlines the design and implementation of the front-end interface, focusing on the **Streamlit** workflow and the interactive elements that facilitate user testing and system transparency.

6.1 Streamlit Workflow & Processing States

To ensure a professional and production-ready user experience, the system utilizes Streamlit's state management to provide real-time feedback during the agentic execution.

- **Processing States:** The UI incorporates st.spinner and st.status containers to notify the user of the active node in the LangGraph workflow . For example, the status shifts from *"Retrieving Document Context..."* to *"Analyzing Information..."* as the state moves through the graph .
- **Response Rendering:** Final answers are displayed using Markdown for readability, with a dedicated section for "Sources" that pulls the page_number metadata from the State Object.

6.2 Clickable Demo Questions & Sidebar

To assist evaluators and streamline the demonstration of the RAG capabilities, the interface includes a persistent sidebar.

- **Clickable Demo Questions:** The sidebar features a set of pre-configured buttons representing common queries related to the uploaded PDF. Clicking a button injects the query directly into the processing pipeline, demonstrating the system's ability to handle expected use cases immediately.
- **System Configuration Display:** The sidebar also reflects the current Model Provider Strategy, indicating whether the system is currently using Groq (Primary) or Gemini (Fallback).

6.3 UI Layout Summary

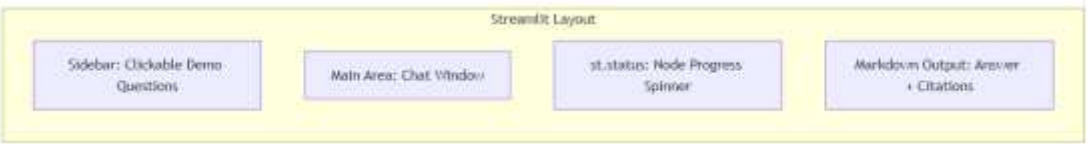

Figure 5 Streamlit Frontend Layout highlighting the sidebar for demo questions and real-time processing status indicators

The interface is structured to prioritize the flow of information while keeping the user informed of the AI's "thought process".

- **Input Layer:** A text area for custom queries and the sidebar for demo questions.
- **Visualization Layer:** Real-time status indicators tracking node transitions.
- **Output Layer:** Grounded answers accompanied by a "Trust Score" (Confidence) and, if applicable, the Escalation Reason.

CHAPTER 7: ERROR HANDLING & OBSERVABILITY

This chapter details the mechanisms used to ensure system stability and the tools employed for developer-level tracing.

7.1 Exception Management & Model Fallbacks

The system is designed with high availability in mind, implementing a dual-model reliability strategy:

- **Primary Provider Failure:** If the Groq API encounters a rate limit or timeout, the exception is caught, and the request is automatically rerouted to the Gemini Fallback .
- **Ingestion Errors:** If the PyPDFLoader fails to extract text from a corrupted or encrypted file, the system raises a user-friendly error message in Streamlit rather than crashing the backend.

7.2 Traceability Hooks (LangSmith)

Observability is integrated via LangSmith, providing a granular audit trail for every query.

- **Pipeline Inspection:** Every state transition in LangGraph is logged, allowing developers to see the exact context retrieved from ChromaDB and the prompt sent to the LLM.
- **Debugging Logic:** Traceability hooks allow for the analysis of the Conditional Routing logic, ensuring that the confidence thresholds are being evaluated accurately

Chapter 8: Conclusion

The Low-Level Design (LLD) for the RAG-Based Customer Support Assistant establishes a robust, modular framework for a localized production-style AI agent. By detailing the internal mechanics of the ChromaDB storage module and the LangGraph orchestration layer, this document ensures that the implementation is both scalable and transparent.The system's strength lies in its Human-in-the-Loop integration, where automated reasoning and confidence-based routing provide a safety net against hallucinations. With a focus on modular prompt management, high-speed inference through Groq, and detailed traceability via LangSmith, this project transitions from a conceptual RAG pipeline into a high-performance, enterprise-ready support solution.

CHAPTER 9: BIBLIOGRAPHY

Official Framework Documentation

- **LangChain & LangGraph:** "LangGraph Documentation: Orchestrating Agentic Workflows." *LangChain Inc.*.
- **ChromaDB:** "Chroma: The AI-Native Open-Source Embedding Database." *Chroma Core Team.* .
- **Google Gemini:** "Gemini API Documentation: Embeddings and Generative Models." *Google AI for Developers.*.
- **Groq Cloud:** "Groq LPU Inference Engine: High-Speed LLM Inference." *Groq Inc.*

Technical References

- **Recursive Character Splitting:** "Text Splitters: Recursive Character Text Splitter Strategy." *LangChain Documentation.*.
- **RAG Best Practices:** "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Meta AI Research.* (2020).
- **Human-in-the-Loop Design:** "Human-in-the-Loop (HITL) Patterns for LLM Agents." *LangChain Design Patterns.*

Observability & Tooling

- **LangSmith:** "Tracing, Debugging, and Monitoring LLM Applications." *LangChain Inc.*
- **Streamlit:** "Streamlit: The fastest way to build and share data apps."*Streamlit.io.*